Pixelfunz: Game Breakdown

Pixelfunz is a D&D-style pixel-art RPG that lives entirely inside a single `index.html` file. There is no asset pipeline, package install, or compile step: the HTML, CSS, JavaScript, UI, game systems, and procedurally drawn pixel art are all shipped together.

> **Build summary:** there is nothing to compile. To run the game, open `index.html` in a modern browser. To work on it, edit that file and refresh the page.

1. What the game is

- A side-scrolling fantasy adventure with a flat overworld and multi-level dungeons.
- A turn-based combat system inspired by tabletop D&D rules.
- Four playable classes: Fighter, Rogue, Wizard, and Cleric.
- A complete run ends when the player defeats the Young Dragon in Dungeon 3.

2. Core player flow

1. Start on the title screen and press Enter.
2. Choose a class, then roll six ability scores using 4d6 drop-lowest.
3. Explore the overworld, open chests, fight monsters, and enter caves.
4. Descend through dungeon levels, gain XP, level up, and unlock stronger class skills.
5. Reach Dungeon 3 and defeat the boss dragon to win.

3. How the world works

Overworld

- The overworld is generated as a flat side-view landscape with grass on a fixed surface row, sky above it, and dirt or rock below it.
- Trees, cave entrances, a town, monsters, and chests are placed across that map after generation.
- The player walks left and right, can climb or hop, and is affected by gravity when unsupported.

Dungeons

- Each dungeon level is carved from solid wall tiles into a sequence of rooms and low corridors.
- Movement remains side-on, so dungeons still use walking, climbing, and gravity instead of a top-down grid.
- Monsters, treasure, stairs, and random ambush encounters increase the risk as depth rises.

4. How combat works

- Combat starts when the player touches a monster or gets ambushed in a dungeon.
- Initiative is rolled first to decide who acts before the other side.
- Basic attacks use a d20 roll plus an attack modifier against enemy Armor Class.
- Natural 20s crit, natural 1s fumble, and class-specific damage logic is layered on top.
- Players can Attack, open the Skills/Spells menu, Defend, Flee, or drink a healing potion.

Class	Role	Shared resource	Skill pattern
Fighter	High durability melee	Stamina	Weapon-heavy burst skills and stuns
Rogue	Accuracy and bonus damage	Tricks	Sneak-focused attacks, advantage, and burst turns
Wizard	Ranged spell damage	Spells	Auto-hit bolts, area-style bursts, and control
Cleric	Balanced offense and sustain	Spells	Radiant attacks plus self-healing

5. Progression and rewards

- Winning fights grants XP and eventually levels.
- Level-ups increase HP and unlock stronger skills for the chosen class.
- Chests award gold, potions, or permanent max-HP boosts.
- The town acts as a recovery stop where the player can restore HP and skill uses and buy potions.

6. Save, UI, and presentation

- The game stores saves in browser localStorage, so save/load is browser-based rather than file-based.
- The canvas renders the world, combat scenes, and menus, while HUD panels and option controls live in the page UI.
- Pixel-art sprites are generated procedurally in code instead of loading external image files.
- Touch controls appear automatically on mobile-sized or touch-enabled devices.

7. How to build or modify it

Running the shipped game

1. Open `index.html` in a modern desktop or mobile browser.
2. Play immediately; there are no dependencies to install.

Working on the code

1. Edit `index.html`; nearly all game logic lives inside the single embedded script.
2. Refresh the browser after each change to test the new behavior.
3. If you want a local server during development, use any simple static file server, but it is optional.

Where to look inside the file

- **HTML/CSS:** page shell, HUD, touch controls, and options modal.
- **Constants and data tables:** tile definitions, class data, skills, monsters, and world dimensions.
- **Generation and state:** overworld creation, dungeon carving, entity placement, and the global game state object.
- **Systems:** save/load, combat, movement, rendering, and input handling.

8. Practical takeaway

If you want to explain Pixelfunz to someone quickly, the simplest summary is: *it is a self-contained browser RPG where one HTML file contains the entire game loop, all art generation, and all rules.* Building it means editing and shipping that one file.